

DSA Interview Preparation (Startups)

Focus: Basics, Arrays & Strings

This file contains **important fundamental problems** frequently asked in startup interviews. Each problem includes:

- Problem statement
 - Approach (intuition + optimal logic)
 - Java solution
 - JavaScript solution
-

1. Two Sum

Problem

Given an array and a target, return indices of two numbers such that they add up to target.

Approach

Use a HashMap to store numbers and their indices.

- Traverse array once
- For each element, check if (target - current) exists
- If yes → return indices

Time Complexity: $O(n)$

Java

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();

    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
}
```

```
    return new int[]{};
}
```

JavaScript

```
function twoSum(nums, target) {
    const map = new Map();

    for (let i = 0; i < nums.length; i++) {
        let complement = target - nums[i];
        if (map.has(complement)) {
            return [map.get(complement), i];
        }
        map.set(nums[i], i);
    }
}
```

2. Maximum Subarray (Kadane's Algorithm)

Problem

Find the contiguous subarray with maximum sum.

Approach

- Keep current sum
- If it becomes negative → reset
- Track maximum

Time Complexity: $O(n)$

Java

```
public int maxSubArray(int[] nums) {
    int max = nums[0], sum = 0;

    for (int num : nums) {
        sum += num;
        max = Math.max(max, sum);
        if (sum < 0) sum = 0;
    }
}
```

```
    return max;
}
```

JavaScript

```
function maxSubArray(nums) {
    let max = nums[0], sum = 0;

    for (let num of nums) {
        sum += num;
        max = Math.max(max, sum);
        if (sum < 0) sum = 0;
    }
    return max;
}
```

3. Best Time to Buy and Sell Stock

Problem

Max profit from one transaction.

Approach

- Track minimum price
- Calculate profit at each step

Time Complexity: $O(n)$

Java

```
public int maxProfit(int[] prices) {
    int min = Integer.MAX_VALUE;
    int profit = 0;

    for (int price : prices) {
        min = Math.min(min, price);
        profit = Math.max(profit, price - min);
    }
    return profit;
}
```

JavaScript

```
function maxProfit(prices) {  
    let min = Infinity;  
    let profit = 0;  
  
    for (let price of prices) {  
        min = Math.min(min, price);  
        profit = Math.max(profit, price - min);  
    }  
    return profit;  
}
```

4. Valid Anagram

Problem

Check if two strings are anagrams.

Approach

- Count characters using array/map
- Compare counts

Time Complexity: $O(n)$

Java

```
public boolean isAnagram(String s, String t) {  
    if (s.length() != t.length()) return false;  
  
    int[] count = new int[26];  
  
    for (int i = 0; i < s.length(); i++) {  
        count[s.charAt(i) - 'a']++;  
        count[t.charAt(i) - 'a']--;  
    }  
  
    for (int c : count) {  
        if (c != 0) return false;  
    }  
}
```

```
    return true;
}
```

JavaScript

```
function isAnagram(s, t) {
    if (s.length !== t.length) return false;

    let count = {};

    for (let ch of s) {
        count[ch] = (count[ch] || 0) + 1;
    }

    for (let ch of t) {
        if (!count[ch]) return false;
        count[ch]--;
    }

    return true;
}
```

5. Longest Substring Without Repeating Characters

Problem

Find length of longest substring without repeating characters.

Approach (Sliding Window)

- Use Set
- Expand right pointer
- Shrink left when duplicate

Time Complexity: $O(n)$

Java

```
public int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int left = 0, max = 0;

    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left));
            left++;
        }
        set.add(s.charAt(right));
        max = Math.max(max, right - left + 1);
    }
    return max;
}
```

JavaScript

```
function lengthOfLongestSubstring(s) {
    let set = new Set();
    let left = 0, max = 0;

    for (let right = 0; right < s.length; right++) {
        while (set.has(s[right])) {
            set.delete(s[left]);
            left++;
        }
        set.add(s[right]);
        max = Math.max(max, right - left + 1);
    }
    return max;
}
```

6. Move Zeroes

Problem

Move all zeros to end while maintaining order.

Approach

- Two pointer technique

Time Complexity: $O(n)$

Java

```
public void moveZeroes(int[] nums) {
    int j = 0;

    for (int i = 0; i < nums.length; i++) {
        if (nums[i] != 0) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
            j++;
        }
    }
}
```

JavaScript

```
function moveZeroes(nums) {
    let j = 0;

    for (let i = 0; i < nums.length; i++) {
        if (nums[i] !== 0) {
            [nums[i], nums[j]] = [nums[j], nums[i]];
            j++;
        }
    }
    return nums;
}
```

7. Reverse String

Problem

Reverse a string.

Approach

- Two pointer swap

Java

```
public void reverseString(char[] s) {  
    int left = 0, right = s.length - 1;  
  
    while (left < right) {  
        char temp = s[left];  
        s[left] = s[right];  
        s[right] = temp;  
        left++;  
        right--;  
    }  
}
```

JavaScript

```
function reverseString(s) {  
    let left = 0, right = s.length - 1;  
  
    while (left < right) {  
        [s[left], s[right]] = [s[right], s[left]];  
        left++;  
        right--;  
    }  
    return s;  
}
```

8. Contains Duplicate

Problem

Check if array contains duplicates.

Approach

- Use Set

Time Complexity: $O(n)$

Java

```
public boolean containsDuplicate(int[] nums) {
    Set<Integer> set = new HashSet<>();

    for (int num : nums) {
        if (set.contains(num)) return true;
        set.add(num);
    }
    return false;
}
```

JavaScript

```
function containsDuplicate(nums) {
    return new Set(nums).size !== nums.length;
}
```

How to Use This File

1. Practice each problem 2-3 times
2. Understand pattern (NOT just code)
3. Try variations
4. Practice explaining out loud (important for interviews)

If you want next level (sliding window advanced / two pointers / recursion), ask me 👍

9. Fibonacci Series

Problem

Print nth Fibonacci number or series.

Approach

- Use iteration (optimal)

- Avoid recursion due to exponential time

Time Complexity: $O(n)$

Java

```
public int fibonacci(int n) {  
    if (n <= 1) return n;  
  
    int a = 0, b = 1;  
    for (int i = 2; i <= n; i++) {  
        int c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

JavaScript

```
function fibonacci(n) {  
    if (n <= 1) return n;  
  
    let a = 0, b = 1;  
    for (let i = 2; i <= n; i++) {  
        let c = a + b;  
        a = b;  
        b = c;  
    }  
    return b;  
}
```

10. Check Prime Number

Problem

Check if a number is prime.

Approach

- Check divisibility up to $\sqrt{n}$

Time Complexity: $O(\sqrt{n})$

Java

```
public boolean isPrime(int n) {  
    if (n <= 1) return false;  
  
    for (int i = 2; i * i <= n; i++) {  
        if (n % i == 0) return false;  
    }  
    return true;  
}
```

JavaScript

```
function isPrime(n) {  
    if (n <= 1) return false;  
  
    for (let i = 2; i * i <= n; i++) {  
        if (n % i === 0) return false;  
    }  
    return true;  
}
```

11. Factorial

Problem

Find factorial of a number.

Approach

- Use iteration (optimal)

Time Complexity: $O(n)$

Java

```
public int factorial(int n) {  
    int result = 1;
```

```
    for (int i = 2; i <= n; i++) {  
        result *= i;  
    }  
    return result;  
}
```

JavaScript

```
function factorial(n) {  
    let result = 1;  
    for (let i = 2; i <= n; i++) {  
        result *= i;  
    }  
    return result;  
}
```

12. Reverse Integer

Problem

Reverse digits of a number.

Approach

- Extract digits using mod (%)

Time Complexity: $O(\log n)$

Java

```
public int reverse(int x) {  
    int rev = 0;  
  
    while (x != 0) {  
        int digit = x % 10;  
        rev = rev * 10 + digit;  
        x /= 10;  
    }  
    return rev;  
}
```

JavaScript

```
function reverse(x) {  
    let rev = 0;  
  
    while (x !== 0) {  
        let digit = x % 10;  
        rev = rev * 10 + digit;  
        x = Math.trunc(x / 10);  
    }  
    return rev;  
}
```

13. Palindrome Number

Problem

Check if a number is palindrome.

Approach

- Reverse number and compare

Time Complexity: $O(\log n)$

Java

```
public boolean isPalindrome(int x) {  
    if (x < 0) return false;  
  
    int original = x, rev = 0;  
  
    while (x != 0) {  
        rev = rev * 10 + (x % 10);  
        x /= 10;  
    }  
    return original == rev;  
}
```

JavaScript

```
function isPalindrome(x) {  
    if (x < 0) return false;  
  
    let original = x, rev = 0;  
  
    while (x !== 0) {  
        rev = rev * 10 + (x % 10);  
        x = Math.trunc(x / 10);  
    }  
    return original === rev;  
}
```

14. Count Digits

Problem

Count number of digits in a number.

Approach

- Divide by 10 repeatedly

Time Complexity: $O(\log n)$

Java

```
public int countDigits(int n) {  
    int count = 0;  
  
    while (n != 0) {  
        count++;  
        n /= 10;  
    }  
    return count;  
}
```

JavaScript

```
function countDigits(n) {  
  let count = 0;  
  
  while (n !== 0) {  
    count++;  
    n = Math.trunc(n / 10);  
  }  
  return count;  
}
```

Now this file covers both:

- Core basics (math + number problems)
- Arrays & strings (most asked in startups)

Next best step: combine these with pattern-based questions 👍